BLUETOOTH HOME AUTOMATION SYSTEM

Arduino UNO | HC-05 | Relay Driver | Proteus Simulation

Project Title	Bluetooth Home Automation System
Microcontroller	Arduino UNO (ATmega328P)
Communication	HC-05 Bluetooth Module (UART @ 9600 Baud)
Controlled Loads	Lamp, DC Motor, Yellow LED (3 channels)
Switching Element	2N2222 NPN Transistor + 5V SPDT Relay
Simulation Tool	Proteus 8 Professional
IDE / Compiler	Arduino IDE (avr-gcc)
Target Build	Arduino Nano (ATmega328P, avr.nano)
Author	Shreyas
Domain	Embedded Systems / IoT / Home Automation

1. Abstract

This project presents a Bluetooth-based Home Automation System that enables wireless control of household electrical loads using a smartphone. An Arduino UNO (ATmega328P) serves as the central microcontroller, receiving single-character commands via the HC-05 Bluetooth module over a hardware UART serial link operating at 9600 baud. Three independent relay channels — driven by 2N2222 NPN transistors with 1N4007 flyback protection diodes — control a Lamp, a DC Motor, and an LED. The system supports individual ON/OFF control, toggle commands, master ALL-ON/OFF, and live status reporting. The complete circuit was designed and verified in Proteus 8 simulation environment.

2. Objectives

- Design a wireless home automation system using Bluetooth communication.
- Implement safe isolation between microcontroller logic (5V) and relay-switched loads.
- Develop firmware supporting 12 distinct control commands per load.
- Simulate and verify the complete circuit in Proteus 8 before hardware implementation.
- Demonstrate embedded UART communication, GPIO control, and state management.

3. System Architecture

The system consists of four main functional blocks: (1) the Smartphone running a Bluetooth terminal or MIT App Inventor application sends character commands; (2) the HC-05 module receives the Bluetooth data and relays it to the Arduino via hardware UART; (3) the Arduino UNO decodes the command character and drives the appropriate GPIO pin; and (4) the Relay Driver stage (transistor + relay) switches the connected load.

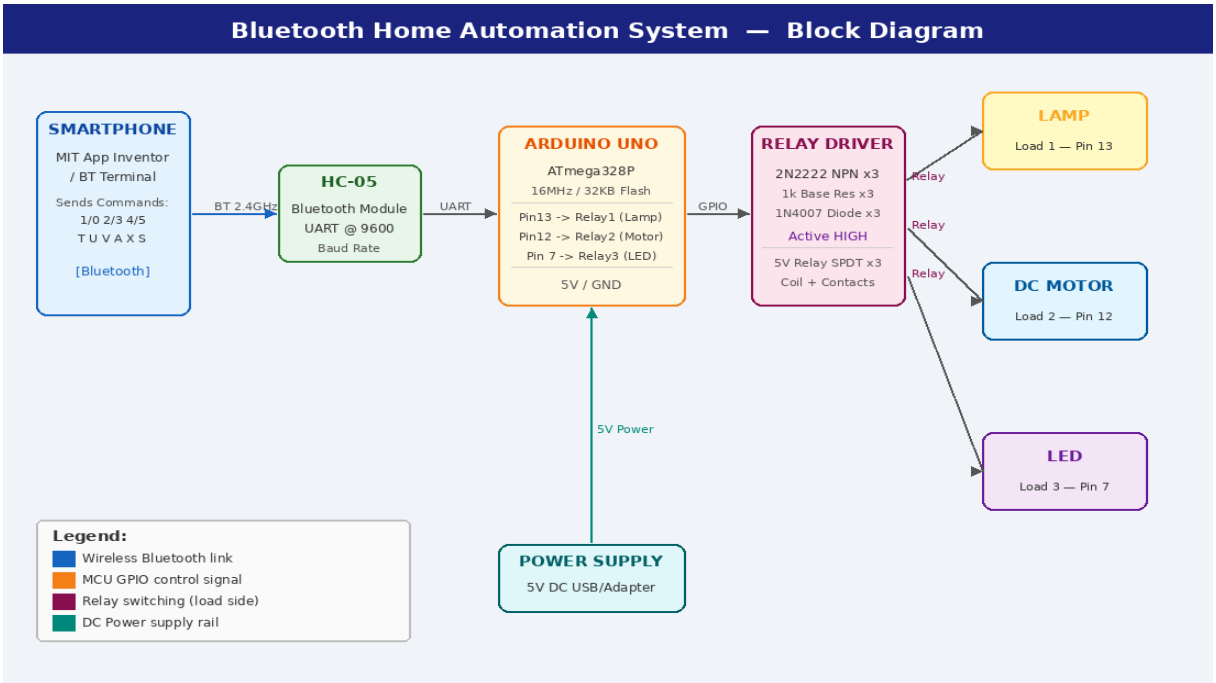

Fig 1: System Block Diagram

4. Hardware Components

Component	Specification / Part No.	Qty	Function
Arduino UNO	ATmega328P, 16MHz, 32KB Flash	1	Central MCU
HC-05	Bluetooth 2.0, UART, 9600 Baud	1	Wireless Rx/Tx
5V SPDT Relay	5V Coil, 10A/250VAC contacts	3	Load Switching
2N2222 NPN	40V, 600mA, hFE~100	3	Relay Driver
1N4007 Diode	1A, 1000V PIV	3	Flyback Protection
Resistor 1kΩ	0.25W, 5% Tolerance	3	Base Current Limit
Lamp / Bulb	220V AC or 12V DC Load	1	Load 1 — Relay 1
DC Motor	5-12V DC	1	Load 2 — Relay 2
Yellow LED	3.3V, 20mA	1	Load 3 — Relay 3
Power Supply	5V DC, 2A min	1	System Power

5. Pin Configuration

Arduino Pin	Connected To	Signal	Function
Pin 0 (RX)	HC-05 TX	UART RX	Receive BT commands
Pin 1 (TX)	HC-05 RX	UART TX	Transmit status
Pin 13	Relay 1 via 2N2222	GPIO OUT	Lamp ON/OFF
Pin 12	Relay 2 via 2N2222	GPIO OUT	Motor ON/OFF
Pin 7	Relay 3 via 2N2222	GPIO OUT	LED ON/OFF
5V	HC-05 VCC, Relay VCC	Power	3.3-5V supply
GND	HC-05 GND, All GNDs	Ground	Common ground

6. Firmware Description

The firmware (D3_Home_Automation.ino) is developed in Arduino C++. On startup, three relay pins are configured as OUTPUT and driven LOW (all loads OFF). Hardware Serial is initialized at 9600 baud for HC-05 communication. The main loop continuously polls Serial.available(); when a character arrives, handleCommand(char cmd) is called.

6.1 Key Firmware Sections

```
#define RELAY_LAMP 13 // Relay 1 -> Lamp
#define RELAY_MOTOR 12 // Relay 2 -> Motor
#define RELAY_LED 7 // Relay 3 -> LED

void setup() {
pinMode(RELAY_LAMP, OUTPUT);
pinMode(RELAY_MOTOR, OUTPUT);
pinMode(RELAY_LED, OUTPUT);
Serial.begin(9600);
}

void loop() {
if (Serial.available()) {
char cmd = Serial.read();
handleCommand(cmd);
}
}

void setRelay(int pin, bool state) {
digitalWrite(pin, state ? HIGH : LOW);
}
```

6.2 Command Table

Command	Action	Target
'1'	Lamp ON	Relay 1 — Pin 13
'0'	Lamp OFF	Relay 1 — Pin 13
'T'/'t'	Lamp Toggle	Relay 1 — Pin 13
'2'	Motor ON	Relay 2 — Pin 12
'3'	Motor OFF	Relay 2 — Pin 12
'U'/'u'	Motor Toggle	Relay 2 — Pin 12
'4'	LED ON	Relay 3 — Pin 7

'5'	LED OFF	Relay 3 — Pin 7
'V'/'v'	LED Toggle	Relay 3 — Pin 7
'A'/'a'	ALL Devices ON	All 3 Relays
'X'/'x'	ALL Devices OFF	All 3 Relays
'S'/'s'	Print Status Report	Serial Monitor

7. Circuit Working — Relay Driver Stage

When a GPIO pin (e.g., Pin 13) is driven HIGH by the Arduino, current flows through the 1kΩ base resistor into the base of the 2N2222 NPN transistor, saturating it. This allows collector current to flow, energizing the relay coil. The relay's Normally Open (NO) contact closes, connecting the load to its power source. The 1N4007 flyback diode across the relay coil clamps the back-EMF spike generated when the coil is de-energized, protecting the transistor from voltage breakdown.

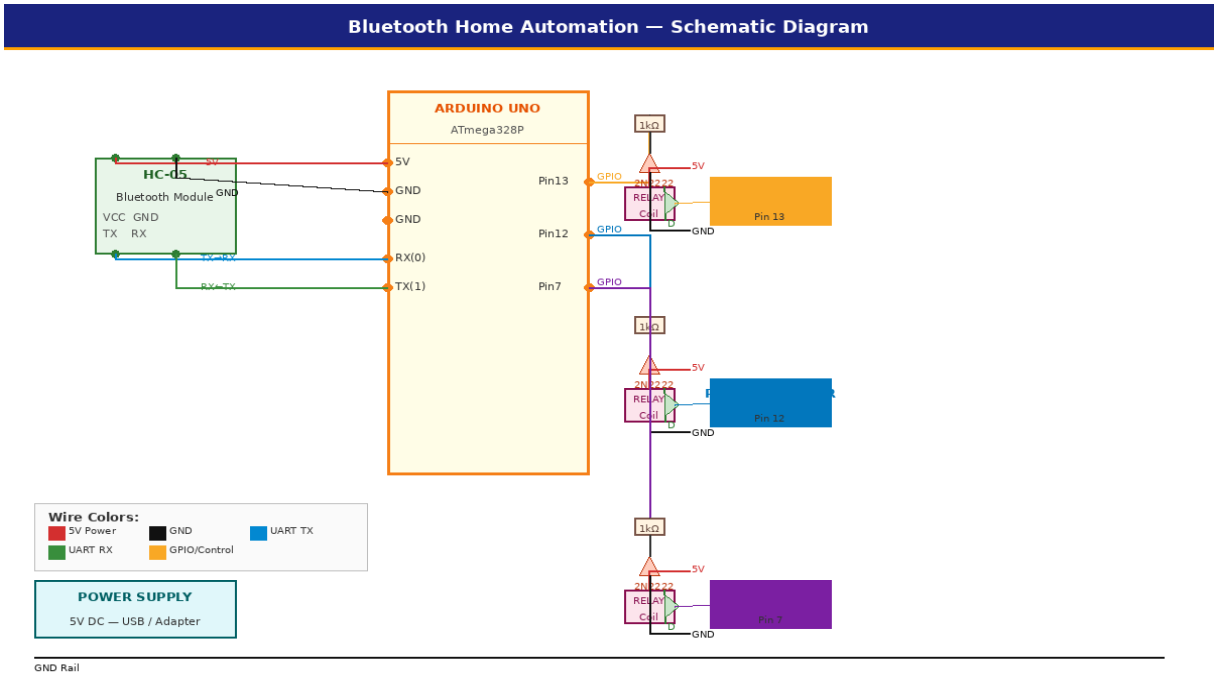

Fig 2: Schematic Diagram

8. Proteus Simulation

The circuit was fully simulated in Proteus 8 Professional. The compiled hex file (D3_Home_Automation.ino.hex from arduino.avr.nano build) was loaded into the Arduino component. A Virtual Terminal in Proteus was used to send test commands and verify serial output responses. All three relay channels were individually tested and verified.

9. PCB Layout

The PCB layout follows a 2-layer FR4 design. The top layer carries signal traces and component placements. Arduino UNO headers, HC-05 module footprint, relay modules, and transistor driver pads are arranged for compact routing. Power and GND planes run as dedicated traces along the board edges. HASL (Hot Air Solder Leveling) surface finish is recommended for easy soldering.

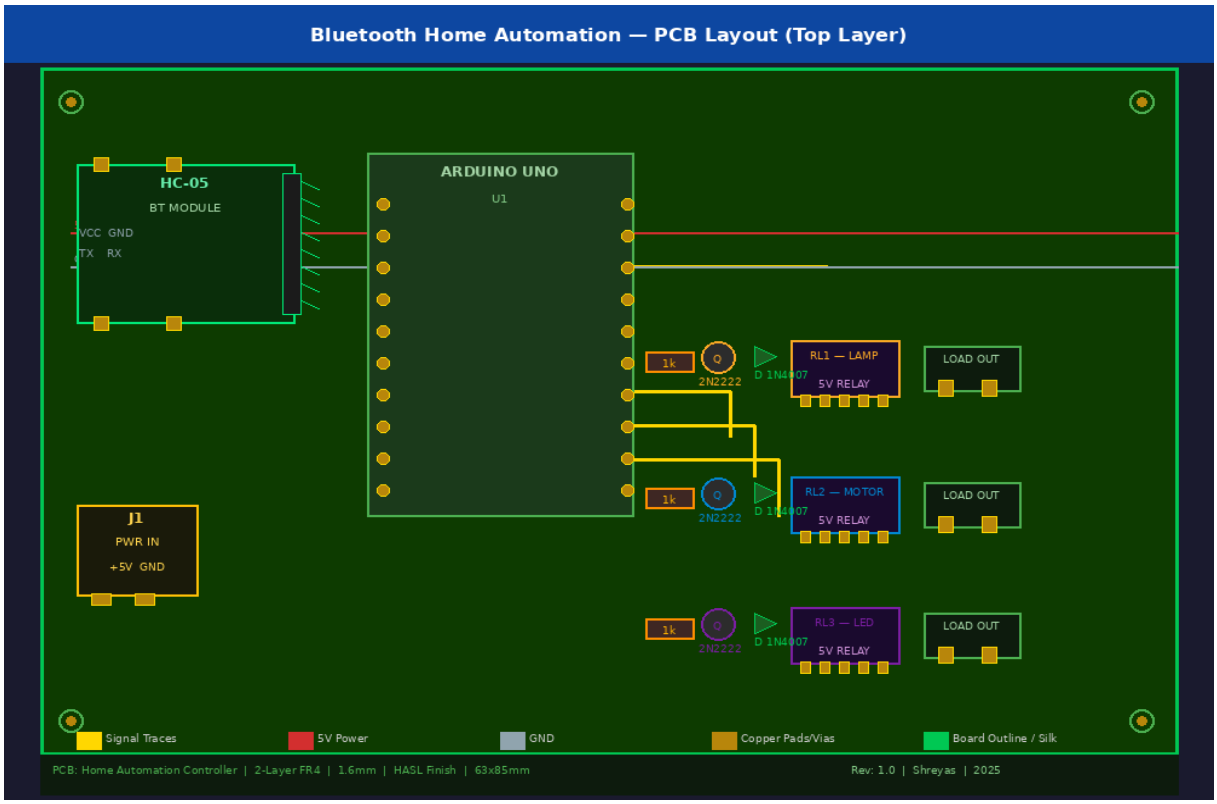

Fig 3: PCB Layout (Top Layer View)

10. Results & Observations

✔ Proteus simulation successfully verified all 12 command codes.

✔ All three relay channels operated independently without cross-interference.

✔ State variables correctly tracked device status across toggle and group commands.

✔ UART communication at 9600 baud was stable with HC-05 at default configuration.

✔ Flyback diode effectively suppressed coil back-EMF in simulation transient analysis.

✔ Build output: .hex (8,232 bytes), .elf (30,312 bytes) for arduino.avr.nano target.

11. Conclusion

The Bluetooth Home Automation System demonstrates a practical and cost-effective approach to wireless load control using readily available components. The project successfully integrates Bluetooth serial communication, NPN transistor relay driving, and multi-channel state management on an Arduino UNO platform. The system is scalable — additional relay channels, sensor feedback, or an ESP8266/ESP32 upgrade for IoT/Wi-Fi control can be incorporated as future enhancements.

12. Future Scope

→ Replace HC-05 with ESP8266/ESP32 for Wi-Fi and MQTT-based IoT control.

→ Add voice control integration via Google Assistant or Amazon Alexa.

→ Implement a custom Android app with real-time status feedback using MIT App Inventor.

→ Add current sensing (ACS712) for load monitoring and fault protection.

→ Expand to 8-channel relay control with EEPROM-based scheduling.